

Classe : GLSI3
Matière : Java Entreprise Edition
Enseignante : Cherifa Nakkach

AU : 2021/2022
Semestre : 1
Nombre de pages : 5

TP Java EE:

ATELIER N°2

Servlet, JSP et HTML

Architecture multi-niveau avec Java EE :



Figure 1 Architecture multi-niveau avec Java EE

Le premier niveau d'une application JEE : l'application web :

Ce premier niveau d'application est constituée de deux éléments clé :

- Les **JSP** (JavaServer Pages) sont des fichiers qui peuvent mélanger Java, HTML et JavaScript. Ces fichiers ont pour but de présenter les données au client. Une JSP est un fichier qui sera transformé en Servlet puis compilé par le conteneur de Servlets.
- Les **Servlets** sont de simple classes Java qui héritent de la classe `HttpServlet` et doivent implémenter deux méthodes : `doPost(...)` , `doGet(...)`. Ces deux méthodes répondent aux deux opérations principales du protocole HTTP : GET et POST.

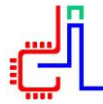
Les JSP (Java Server Pages) :

Les JSP (Java Server Pages) correspondent à la partie vue (web) d'une application JEE. Ces pages sont dynamiques et peuvent contenir du code Java. Les JSP sont converties en classes Java par le conteneur de Servlets (par exemple Tomcat).

Il existe plusieurs solutions pour écrire du code Java dans une JSP. Basiquement, il suffit de le placer entre les balises . On parle de scriptlet. Pour évaluer une expression (variable Java par exemple) il faut par contre utiliser la balise.

Contrôleurs, Servlets

Une servlet est une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Ces données sont le plus généralement présentées au format HTML, mais elles peuvent également l'être au format XML, ou tout autre format destiné aux navigateurs web. Les servlets utilisent l'API Java Servlet (package `javax.servlet`). Une servlet s'exécute dynamiquement sur le serveur web et permet l'extension des fonctions de ce dernier, typiquement : accès à des bases de données, transactions d'e-commerce, etc. Une servlet peut être chargée automatiquement lors du démarrage du serveur web ou lors de la première requête du client. Une fois chargées, les servlets restent actives dans l'attente d'autres requêtes du client. L'utilisation de servlets se fait par le biais d'un conteneur de servlets côté serveur. Celui-ci constitue l'environnement d'exécution de la servlet et lui permet de persister entre les requêtes des clients. L'API définit les relations entre le conteneur et la servlet. Le conteneur reçoit la requête du client, et sélectionne la servlet qui aura à la traiter. Le conteneur fournit également tout un ensemble de services standards pour simplifier la gestion des requêtes et des sessions.



Activité 1

1. Toujours dans le même workspace que TP1, créer un projet « *Dynamic Web Project* » appelé *TP2_JEE*,
2. Créer dans le dossier WebContent le fichier *login.html*.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="windows-1256">
<title>Insert title here</title>
</head>
<body>
<form action="vue.jsp" method="post">
  Login: <input type="text" name="login"><br/>
  Password: <input type="password" name="password"><br/>
  <input type="submit" value="OK">
</form>
</body>
</html>
```

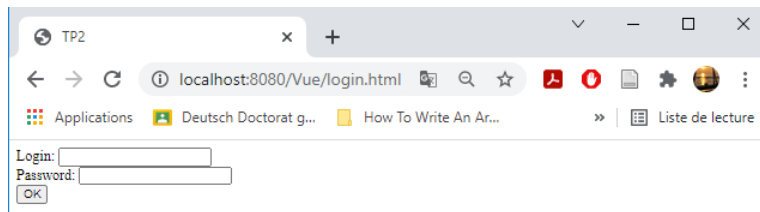
3. Créer dans le dossier WebContent le fichier *vue.jsp*

```
<%@ page language="java" contentType="text/html; charset=windows-1256"
    pageEncoding="windows-1256"%>

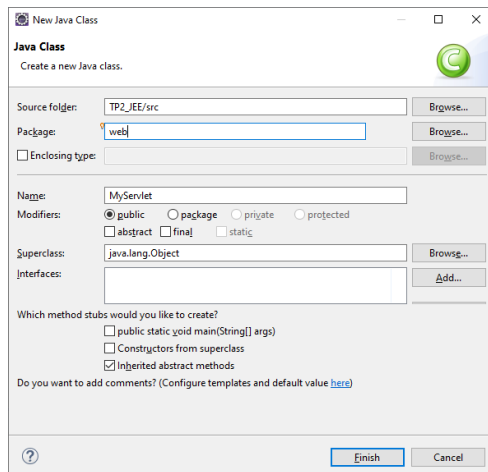
<%
String L = request.getParameter("login"); String
p = request.getParameter("password");

%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=windows-1256">
<title>Insert title here</title>
</head>
<body>
  <h3>Votre login est : <% out.print(L); %></h3>
  <h3>Votre password est : <%=p %></h3>

</body>
</html>
```



4. Créer une Servlet appelée MyServlet, dans le package web,



package web;

```
import java.io.IOException;
import javax.servlet.ServletException; import
javax.servlet.annotation.WebServlet; import
javax.servlet.http.HttpServlet; import
javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```
@WebServlet (name="ms",urlPatterns= {"/controleur"})
public class MyServlet extends HttpServlet{
```

```
    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException { String
        L=request.getParameter("login");
        String p=request.getParameter("password");
        request.setAttribute("login", L);
        request.setAttribute("password", p);

        request.getRequestDispatcher("vue.jsp").forward(request,response);
    }
}
```

```
}
```

5. Modifier le fichier vue.jsp comme suit :

```
<%@ page language="java" contentType="text/html; charset=windows-1256"
    pageEncoding="windows-1256"%>

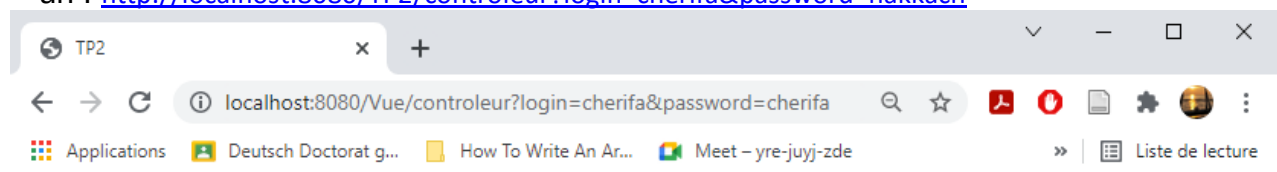
<%
    String L = (String) request.getAttribute("login"); String p =
    (String) request.getAttribute("password");

%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=windows-1256">
<title>Insert title here</title>
</head>
<body>
    <h3>Votre login est : <% out.print(L); %></h3>
    <h3>Votre password est : <%=p %></h3>

</body>
</html>
```

6. Tester votre travail :

url : <http://localhost:8080/TP2/controleur?login=cherifa&password=nakkach>



Votre login est : cherifa

Votre password est : cherifa

Activité 2 :

Le but de cette activité est de créer un petit projet Web permettant de créer un contact, de l'afficher, puis, de le modifier.

Pour cela, nous allons créer deux fichiers JSP responsables de l'affichage d'un contact et de son édition (ViewContact.jsp et EditContact.jsp). Chacune de ces JSP communiquera avec une classe Java qui aura le rôle de contrôleur : le Servlet (ContactServlet.java).

1. Ajoutez un fichier HTML (index.html) dans le dossier src/main/webapp. Editez-le afin d'afficher une image de votre choix, ainsi qu'un lien nommé Ajouter Contact vers l'url ./EditContact.jsp.
2. Ajoutez du code Javascript pour afficher la date du jour.
3. Re-déployez un nouveau WAR de votre application vers Tomcat.
4. Visitez la page HTML que vous venez de créer avec votre navigateur.

La JSP EditContact

1. Créez un nouveau fichier JSP dans /src/main/webapp que l'on appellera EditContact.jsp.
2. Modifiez cette JSP afin qu'elle affiche un formulaire permettant d'insérer un nom, un numéro de téléphone, et une date de naissance.
3. Modifiez la JSP pour que le champ date de naissance soit par défaut égal à la date du jour (utilisez Javascript).
4. Testez dans votre navigateur.

5. Remplacez le code Javascript par du code Java.
6. Quelle est la différence entre le code Java et le code Javascript ?

Les Servlets

Créer une nouvelle servlet avec Eclipse

1. Ajoutez une nouvelle servlet au projet : ContactServlet.
2. À quoi sert l'annotation @WebServlet, les méthodes doGet et doPost ?
3. Le protocole HTTP définit 4 types d'opérations : GET, POST, DELETE, PUT. Quel est l'utilité de PUT et DELETE ? Comment les implémenter avec une Servlet ?

Test de la nouvelle servlet

Un objet de type PrintWriter peut être récupéré depuis l'objet response afin d'écrire des chaînes de caractères dans la réponse d'une servlet.

1. Créez une classe Contact (name, email, phone, birthday).
2. Modifiez la méthode doGet(...) afin d'afficher les informations d'une instance d'un contact.
3. Testez votre toute nouvelle Servlet

Communication entre une Servlet et une JSP

Transférer la requête vers une JSP

Il est possible, grâce à l'objet RequestDispatcher accessible via la méthode request.getRequestDispatcher de l'objet request, de transférer la requête et la réponse de la servlet vers une JSP.

1. Modifiez la méthode doGet afin de rediriger l'utilisateur vers la JSP EditContact.
2. Testez.
3. Modifiez la méthode doPost afin de récupérer les paramètres du formulaire. On pourra utiliser la méthode getParameter de l'objet request.
4. Il est possible de passer des paramètres à une JSP en utilisant la méthode request.setAttribute(String e, Object o). Avec e le nom du paramètre et o sa valeur. Puis, du côté JSP on pourra récupérer ce paramètre en utilisant la méthode request.getAttribute(String e). Utilisez ces méthodes afin d'afficher le contact créé par la méthode doPost(...) dans une nouvelle JSP ViewContact.jsp.
5. Testez.

Activité 3 : Culture Java EE

1. Qu'est-ce qu'un conteneur de servlet ? Citez un exemple.
2. Qu'est-ce qu'un fichier WAR ?
3. Qu'est-ce que le fichier web.xml ? À quoi sert-il ?
4. Quelle est la différence entre une JSP et une Servlet ?
5. Quels sont les attributs accessibles en Java depuis une JSP ?
6. À quoi sert la session HTTP ?
- 7- Quelle est la différence entre JSF et JSP ?
8. Qu'est-ce que Struts et Spring MVC ?
9. Quelle est la différence entre MVC et MVC2 ? Quels sont les avantages et inconvénients de l'un par rapport à l'autre ?